Practical No. 5

Title: Monitoring Model Drift Using MLflow and Evidently

Aim

To monitor the performance of a trained machine learning model on new production data and identify model/data drift using MLflow and Evidently.

Theory

In a traditional machine learning project, a model is trained using historical data and then deployed to make predictions on new data. However, the characteristics of real-world data can change over time. As a result, a model that initially performs well may gradually become less accurate. This phenomenon is known as model drift.

Model drift refers to a situation where the performance or behavior of a machine learning model changes after deployment because the real-world environment or incoming data has changed. Drift can occur due to changes in customer behavior, environmental conditions, business processes, sensor readings, or other external factors.

For example, consider a machine learning model developed to predict hardware failures in telecom towers. The model may be trained using historical values of voltage, temperature, and signal drop rate. If environmental conditions change and the new tower data contains significantly higher temperatures and different voltage patterns, the model may no longer make accurate predictions.

Tools Required

- **Python / Python IDLE**
- **Existing `telecom_tower_model.pkl`**
- **Existing `tower_telemetry.csv`**
- **MLflow**
- **Evidently**
- **Pandas**
- **Scikit-learn**

Step 1: Create Practical 5 Folder

C:\Users\sarve\Desktop\MLOPS\mlops prac\prac_5

The folder should initially contain:

prac_5

├── telecom_tower_model.pkl

└── tower_telemetry.xlsx

├── new_tower_telemetry.xlsx

Step 2: Open Command Prompt

Go to your practical folder:

cd "C:\Users\sarve\Desktop\MLOPS\mlops prac\prac_5"

Step 3: Install Required Libraries

Run:

pip install pandas scikit-learn joblib mlflow evidently

Step 5 — Check Your Old Model

Before we write the monitoring program, we need to make sure your `.pkl` model was trained using the same 8 features in your dataset.

Your model should have been trained with:

Temperature_C

Battery_Voltage

Power_Consumption_W

Signal_Strength_Percent

Fan_Speed_RPM

Humidity_Percent

Traffic_Load

Tower_Age_Years

and the target:

Failure_Within_48Hrs

Step 6 — Create `monitor_model.py`

Open Python IDLE.

Click:

File → New File

Save it as:

Monitor_model.py

Step 7 — Paste This Code

Paste the following into `monitor_model.py`:

```
import pandas as pd
import joblib
import mlflow
from sklearn.metrics import accuracy_score
# ---------------------------------------
# 1. Load the trained Random Forest model
# ---------------------------------------
```

```
model = joblib.load("telecom_tower_model.pkl")
print("Model loaded successfully!")
# -------------------------------------
# 2. Load NEW production data
# -------------------------------------
new_data = pd.read_excel("new_tower_telemetry.xlsx")
print("New production data loaded successfully!")
# -------------------------------------
# 3. Select ML features
# -------------------------------------
features = [
    "Temperature_C",
    "Battery_Voltage",
    "Power_Consumption_W",
    "Signal_Strength_Percent",
    "Fan_Speed_RPM",
    "Humidity_Percent",
    "Traffic_Load",
    "Tower_Age_Years"
]
X_new = new_data[features]
```

```
y_new = new_data["Failure_Within_48Hrs"]

# ---------------------------------------

# 4. Make predictions

# ---------------------------------------

predictions = model.predict(X_new)

# ---------------------------------------

# 5. Calculate accuracy

# ---------------------------------------

accuracy = accuracy_score(

    y_new,

    predictions

)

print("----------------------------------")

print("MODEL DRIFT MONITORING")

print("----------------------------------")

print(

    "Production Accuracy:",

    round(accuracy, 2)

)

# ---------------------------------------
```

```
# 6. Start MLflow experiment
# ------------------------------------
mlflow.set_experiment(
    "Telecom_Tower_Model_Monitoring"
)
with mlflow.start_run():
    mlflow.log_param(
        "model",
        "Random Forest"
    )
    mlflow.log_param(
        "n_estimators",
        100
    )
    mlflow.log_metric(
        "production_accuracy",
        accuracy
    )
    print("Results logged successfully to MLflow")
```

```
# ------------------------------------
# 7. Drift / performance threshold
# ------------------------------------
threshold = 0.80
if accuracy < threshold:
    print()
    print("WARNING: Model performance has degraded!")
    print("Model retraining is recommended.")
else:
    print()
    print("Model performance is stable.")
```

Step 8 — Run the Program

Expected output

If everything is correct, you should see:

Model loaded successfully!
New production data loaded successfully!

MODEL DRIFT MONITORING

Production Accuracy: 0.xx

Results logged successfully to MLflow

WARNING: Model performance has degraded!
Model retraining is recommended.

Step 9 — Start MLflow

Now open a **second Command Prompt window**.

Run:

cd "C:\Users\sarve\Desktop\MLOPS\mlops prac\prac_5"

Then:

mlflow ui

You should see something similar to:

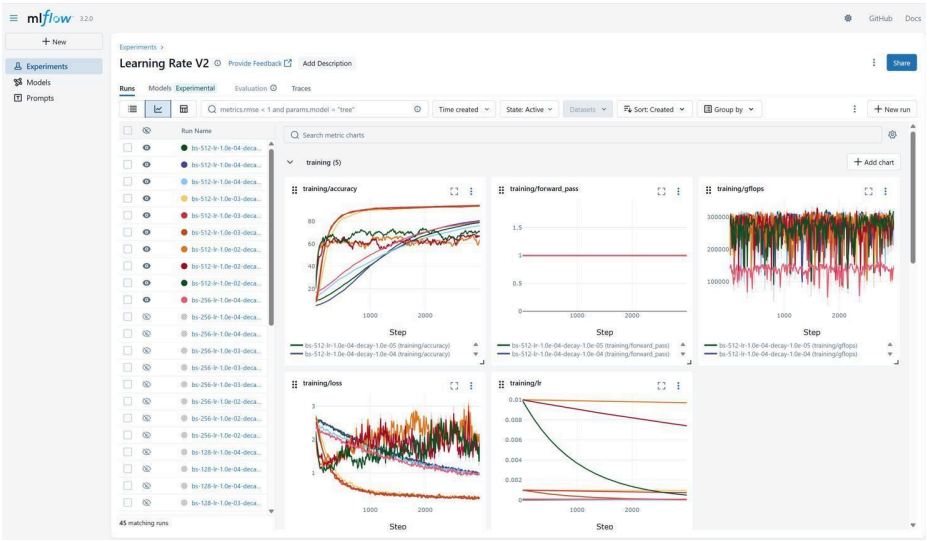
Listening at: http://127.0.0.1:5000

Step 10 — Open MLflow

Open Chrome and enter:

http://127.0.0.1:5000

You should see the MLflow dashboard.



Look for:

Telecom_Tower_Model_Monitoring

Click it.

You should see your experiment/run.

Step 11 — Check the MLflow Metrics

Inside the run, you should find:

Parameters

model = Random Forest

n_estimators = 100

Metric

production_accuracy = 0.xx

This demonstrates that **MLflow is tracking the model's production performance**.